

Sistemas Operativos

Parcial 2 — 27/06/2024 · Resolución comentada

Material de estudio — desarrollo paso a paso

PRÁCTICA

Ejercicio 1 — Segmentación (7 p)

Enunciado: tabla (Segmento, Marco): (0,2),(1,-),(8,0),(2,-),(3,1),(6,3),(7,-),(4,-),(5,-). Cada segmento/marco mide 1024 bytes y el marco 0 comienza en la dirección física 0. Hallar las direcciones físicas de varias direcciones virtuales.

Idea clave Dirección física = Marco $\times$ 1024 + desplazamiento, válido solo si el segmento está presente y el desplazamiento está entre 0 y 1023.

Dir. virtual	Resolución	Dir. física / resultado
a) (1, 0)	Segmento 1 no está en memoria	Fallo de segmento (no presente)
b) (2, 1000)	Segmento 2 no está en memoria	Fallo de segmento (no presente)
c) (6, 100)	$3 \times 1024 + 100 = 3072 + 100$	3172
d) (0, 5)	$2 \times 1024 + 5 = 2048 + 5$	2053
e) (8, 12)	$0 \times 1024 + 12$	12
f) (6, 1042)	$1042 > 1023 \rightarrow$ fuera de límite	Error de límite (trap)
g) (8, 1024)	$1024 > 1023$ (último byte válido = 1023)	Error de límite (trap)

Nota: (a) y (b) fallan porque el segmento no fue cargado; (f) y (g) fallan por violación de límite del segmento, que es un error distinto.

Ejercicio 2 — Segmentación-paginación (10 p)

Datos: espacio lógico 16 GB, página 2 KB, hasta 512 segmentos, descriptores de 4 bytes. Hallar el tamaño de la tabla de páginas de un segmento.

Idea clave Todo a potencias de 2. Los bits totales menos los de segmento y los de desplazamiento dan los bits del número de página dentro del segmento.

- 16 GB = 2^{34} $\rightarrow$ 34 bits totales.
- 512 segmentos = 2^9 $\rightarrow$ 9 bits de segmento.
- Página 2 KB = 2^{11} $\rightarrow$ 11 bits de desplazamiento.

Bits de página por segmento: $34 - 9 - 11 = 14 \rightarrow 2^{14} = 16384$ páginas.

Tabla de páginas = $2^{14} \times 4$ bytes = 65536 bytes = 64 KB

Ejercicio 3 — Paginación bajo demanda (16 p)

Cadena de referencias: 5 2 1 0 5 2 4 5 2 1 0 4 5 2 1 0 5 2 4 5 2 1 0 4

a) Con 3 marcos — fallos (F) / aciertos (·)

Ref	5	2	1	0	5	2	4	5	2	1	0	4	5	2	1	0	5	2	4	5	2	1	0	4
FIFO	F	F	F	F	F	F	F	·	·	F	F	·	F	F	F	F	F	F	F	·	·	F	F	·
LRU	F	F	F	F	F	F	F	·	·	F	F	F	F	F	F	F	F	F	F	·	·	F	F	F

Ref	5	2	1	0	5	2	4	5	2	1	0	4	5	2	1	0	5	2	4	5	2	1	0	4
Óptima	F	F	F	F	.	.	F	.	.	F	F	.	.	F	F	.	.	F	F	.	.	F	F	.

- **FIFO:** 18 fallos — expulsa la página cargada hace más tiempo.
- **LRU:** 20 fallos — expulsa la menos usada recientemente; en este bucle rinde peor que FIFO porque descarta justo la que se va a reutilizar.
- **Óptima:** 13 fallos — expulsa la que tardará más en volver a usarse (cota mínima teórica).

b) Con 4 marcos — ¿anomalía de Belady?

Idea clave La anomalía de Belady solo aparece en FIFO (no en algoritmos de pila como LRU/Óptima). Basta rehacer FIFO con 4 marcos y comparar.

Ref	5	2	1	0	5	2	4	5	2	1	0	4	5	2	1	0	5	2	4	5	2	1	0	4
FIFO (4)	F	F	F	F	.	.	F	F	F	F	F	F	F	F	F	F	.	.	F	F	F	F	F	F

Comparación: 3 marcos → 18 fallos; 4 marcos → 20 fallos. Al aumentar los marcos los fallos aumentaron (18 → 20), por lo tanto **sí se produce la anomalía de Belady**.

Ejercicio 4 — ¿Entra el programa? (16 p)

Datos: espacio de 16 bits ($2^{16} = 65536$ bytes). Código 32768 B, datos 16386 B, pila 15871 B. Cada sección se redondea a páginas completas.

Idea clave No basta sumar bytes: cada sección ocupa un número entero de páginas (fragmentación interna), y por eso hay que redondear hacia arriba cada una.

a) Páginas de 2048 bytes → 32 páginas disponibles

Sección	Bytes	Cálculo	Páginas
Código	32768	$32768 / 2048 = 16$ exacto	16
Datos	16386	$16384 + 2 \rightarrow$ se pasa por 2 B	9
Pila	15871	entre 7 y 8 páginas	8
Total			33

Necesita 33 páginas y solo hay 32 → **No entra**. (La suma en bytes es $65025 < 65536$, pero el redondeo lo deja afuera por 1 página.)

b) Páginas de 256 bytes → 256 páginas disponibles

Sección	Bytes	Cálculo	Páginas
Código	32768	$32768 / 256 = 128$ exacto	128
Datos	16386	$16384 + 2 \rightarrow 64,03 \rightarrow 65$	65
Pila	15871	$256 \times 62 = 15872 \rightarrow 62$	62
Total			255

Necesita 255 páginas y hay 256 → **Sí entra** (justo, con una página de sobra). Con páginas más chicas se desperdicia menos por fragmentación interna.

Ejercicio 5 — Paginación de dos niveles (20 p)

Datos: máximo direccionable 512 MB, página 16 KB, direccionamiento a byte. Tabla de primer nivel de 512 entradas.

Páginas máximas por proceso

- 512 MB = 2^{29} bytes; página 16 KB = 2^{14} bytes.

$$\text{Máx. páginas} = 2^{29} / 2^{14} = 2^{15} = 32768 \text{ páginas}$$

Tablas de segundo nivel para 16421 páginas

- Primer nivel: 512 = 2^9 entradas. Cada tabla de 2.º nivel cubre $2^{15} / 2^9 = 2^6 = 64$ páginas.

$$\lceil 16421 / 64 \rceil = \lceil 256,58 \rceil = 257 \text{ tablas de segundo nivel}$$

Justificación: $64 \times 256 = 16384$ páginas llenan 256 tablas; quedan 37 páginas que exigen 1 tabla más $\rightarrow 257$.

Ejercicio 6 — FAT (15 p)

a) Reconstrucción del archivo

Idea clave Un inicio de archivo es un clúster al que nadie apunta en la columna SIGUIENTE. Aquí son el 9 y el 12.

- Cadena desde 9: $9 \rightarrow 20 \rightarrow 17 \rightarrow 19 \rightarrow 18 \rightarrow 16 \rightarrow 21 \dots$ apunta al clúster 21, fuera de los datos dados $\rightarrow$ no reconstruible.
- Cadena desde 12: $12 \rightarrow 13 \rightarrow 10 \rightarrow 11 \rightarrow 14 \rightarrow 15 \rightarrow \text{EOF}$ $\rightarrow$ termina limpio y usa solo clústeres disponibles.

Orden	12	13	10	11	14	15
Contenido	0	4	C	B	2	F

La información del archivo es **04CB2F** (6 clústeres). La otra cadena continúa en el clúster 21, del que no hay contenido.

b) Clústeres máximos con FAT de 16 bits

$$2^{16} = 65536 \text{ clústeres}$$

Máximo teórico. En la práctica se reservan valores (0x0000 libre, EOF, defectuoso) y quedan ~ 65524 utilizables.

c) Tamaño de clúster y sectores por clúster

$$\text{Clúster} = 2048 \text{ MB} / 65536 = 2^{31} / 2^{16} = 2^{15} = 32768 \text{ B} = 32 \text{ KB}$$

$$\text{Sectores por clúster} = 32768 / 512 = 64 \text{ sectores}$$

d) Espacio ocupado por el archivo

$$6 \text{ clústeres} \times 32 \text{ KB} = 192 \text{ KB}$$

Ejercicio 7 — i-nodos (16 p)

Datos: bloque 8 KB; i-nodo con 5 índices directos, 2 indirectos simples y 2 indirectos dobles; cada bloque de índices tiene 32 apuntadores.

a) Bloques de datos máximos

$$5 + 2 \times 32 + 2 \times 32 \times 32 = 5 + 64 + 2048 = 2117 \text{ bloques de datos}$$

b) Bloques de enlaces (índices) al máximo

- Indirectos simples: 2 bloques de índices.
- Indirectos dobles: cada uno 1 bloque de 1.er nivel + 32 de 2.º = 33; $\times 2 = 66$.

$$2 + 66 = 68 \text{ bloques de índices}$$

c) Tamaño máximo del archivo

$$2117 \times 8 \text{ KB} = 16936 \text{ KB} \approx 16,54 \text{ MB} (17\,342\,464 \text{ bytes})$$

d) Espacio ocupado en disco (al máximo)

$$(2117 + 68) \times 8 \text{ KB} = 2185 \times 8 \text{ KB} = 17480 \text{ KB} \approx 17,07 \text{ MB}$$

TEORÍA — Verdadero / Falso

Recordá: cada acierto +3, cada error -1. Conviene dejar en blanco lo dudoso.

#	Resp	Justificación
1	F	La contigua es la simple y la particionada; la paginada es dispersa.
2	V	Paginación y segmentación son asignación dispersa.
3	V	La paginación genera fragmentación interna en la última página.
4	V	Cada partición fija aloja un solo proceso.
5	V	La segmentación (segmentos variables) genera fragmentación externa.
6	V	Sin memoria virtual todo el proceso debe estar en memoria.
7	V	Tabla invertida/hash: una entrada por marco físico. (Ver nota).
8	F	La tabla invertida es única y global, no una por proceso.
9	F	Elimina la externa, pero queda la interna en la última página.
10	V	Los segmentos pueden tener tamaños distintos.
11	V	La segmentación paginada elimina la fragmentación externa.
12	V	La memoria virtual permite programas mayores que la RAM física.
13	F	FIFO expulsa la que estuvo más tiempo en memoria (la más antigua).
14	V	Belady ocurre en FIFO y no en Óptima. (Ver nota).
15	F	Óptima no es implementable: requiere conocer el futuro.
16	F	El bit de validez/presencia indica si la página está en memoria, no si se accedió recientemente.
17	V	Un archivo puede verse como bytes, registros o registros indexados.
18	V	En Ext2 el ejecutable se identifica por el bit de permiso de ejecución.
19	V	En FAT el ejecutable se reconoce por la extensión.
20	V	En FAT el nombre está en la entrada de directorio; renombrar se hace ahí.
21	F	El acceso puede ser secuencial o directo, no siempre directo.
22	F	La asignación indexada no produce fragmentación externa.
23	V	La asignación contigua sí produce fragmentación externa.
24	V	En FAT la entrada de directorio guarda el clúster inicial.
25	F	FAT es una variante de asignación enlazada, no indexada.
26	F	En Ext2 los bloques índice se crean bajo demanda, no todos al crear.
27	F	Un clúster es un conjunto de sectores, no de pistas.
28	F	El clúster es potencia de 2 (512 B, 1 K, 2 K, 4 K...); 6 KB no es válido.
29	F	En Ext2 los bloques de índices y de datos tienen el mismo tamaño.
30	F	El nombre está en el directorio; renombrar no requiere tocar el i-nodo.
31	V	En directorio de un solo nivel hay conflictos de nombres.

#	Resp	Justificación
32	V	El mapa de bits es ineficiente si no se mantiene en memoria.
33	V	Capacidad = cabezas × cilindros × sectores/pista × tamaño de sector.

Ítems para revisar con tu apunte (penalización –1)

- **#7:** lo tomé como V interpretándolo como tabla invertida (una entrada por marco). Si tu cátedra distingue estrictamente 'tabla hash' de 'tabla invertida', podría ser F.
- **#14:** lo tomé como V porque atribuye Belady a FIFO. El matiz: en Óptima nunca ocurre (no 'casi nunca'). Si tu profe es literal con el 'casi nunca', podría marcarlo F.